

\Ella Potter & James Whipple

Professor Beichel

ECE:3360 Embedded Systems

Post-Lab Report 4

1. Introduction

The main goal for this lab was to control a cooling fan using Timer/Counter0's waveform generation functionality. The fan should be turned on by the push of a push-button switch. Once turned on, the fan is adjustable by turning an RPG, which will ultimately change the duty cycle of the PWM signal being sent to the fan, and will increase or decrease the fan's speed depending on which way the RPG was turned. The fan's on/off status and duty cycle percent will all be displayed to the user on an LCD and will be constantly updated as the user interacts with the system.

2. Schematic

Figure 1 shows the circuit set-up for the RPG (rotary pulse generator), the PBS (pushbutton switch), the LCD(liquid crystal display), and the ATmega328P microcontroller. For this circuit, the microcontroller contains 3 inputs and 7 outputs. For the inputs, it will receive two signals from the RPG on pins PC0 and PC1 and one signal from the PBS on pin PB0. For outputs, two outputs are used to control the LCD, Enable and RS, these are connected to PB4 and PB5. The other 4 outputs to the LCD will send the commands/data and are connected to PD0-PD3. Finally, the last output is PD5(OC0B) which will send the PWM signal to the fan.

3. Discussion

For this Lab, we had two new hardware components to integrate into our circuit as well as two components used in other labs.

- ❖ **Push-Button Switch:** used to control the fans power
 - Goes through a debounce circuit before reaching uC input
 - Pull-up resistor and low-pass filter
 - When pushed fan should turn on/off immediately
- ❖ **Rotary Pulse Generator:** used to change the duty cycle of the PWM signal, which will change the fans speed
 - Uses same debounce circuit as PBS
 - Pull-up resistor and low-pass filter
 - Has two pins A and B both sending signals asynchronously
 - CW: 00 -> 01 -> 11 -> 10 -> 00
 - CCW: 00 -> 10 -> 11 -> 01 -> 00

- When turned clockwise duty cycle will increase, and counterclockwise duty cycle will decrease
- ❖ **Cooling Fan:** simple fan that spins at a certain speed based on PWM signal from microcontroller
 - Connected to microcontrollers OC0B so that it can receive the PWM signal generated by Timer/Counter0
 - Voltage supply must be 9V wall-wart that can be plugged into uC and then received through the Vin pin
 - Tachometer signal not used but could be used to determine the overall fan speed
- ❖ **LCD:** used to display the fans power status and well as the current duty cycle percentage to the user
 - Vee is connected to a contrast voltage using a pull-up and pull-down resistor
 - The RS pin is used to select the register to read/write to
 - Input of 0 : instruction register selected, data is treated as commands
 - Input of 1: data register selected, data is treated as characters
 - The R/W pin is used to select whether to read data from the LCD or write data to the LCD
 - Since we are only writing, it is connected to ground for connection 0
 - The E pin is used to enable reading/writing to the LCD
 - To write, we need to strobe the signal from high to low
 - The D0-7 pins are data-bus lines and carry the data that is sent to the register
 - D0-D3 are the low-order bits and we will not use
 - D4-D7 are the high-order bits and the ones we use to send characters

3.2 Software

After setting up the circuit, the logic for the microcontroller was written using AVR assembly language. The following table explains the different loops and subsections of the code.

Code Section	Description	Use
Start	This section first initializes all the inputs and outputs of the uC by using cbi and sbi calls and the data direction registers. It will then initialize the LCD by	This code segment is only ran once

	calling that subroutine. After initializing, it will wait 10ms and then send the original fan status(off) and original duty cycle(0) to the LCD to be displayed. This loop also takes care of initializing the pin-change registers to make sure they are ready for user interaction.	at the very beginning of execution.
Loop	This loop is the main loop that the program will run until a pin-change is detected. It will wait 30 ms and then call the subroutine that updates the LCD and then continue to loop. This allows the LCD to be constantly updating so that when an interrupt causes the fan status to change or the DC to change the LCD will also be updated with the new information.	This code is continuously ran until an interrupt occurs.
Update_LCD_DC	This routine will update the LCD's display by first clearing the display and then re-writing the message with the dc and fan status. The "DC= %" and "Fan:" strings are both constant strings that are stored in memory to be easily accessed each time. Where as the dc and fan power status are both stored in registers so that they can be constantly changed when needed. Another key part in displaying the DC is making sure that we convert it to a correct percentage. This will be shown below in Timer calculations.	This routine is called every 300 ms allowing the LCD to be constantly refreshing its display
RPG_intr	This routine is in charge of handling pin changes on the C register, where the RPG is connected. If pin change was detected, the program will determine which way the PRG turned. It does this by checking if the pin change was a falling edge and then checking the state of the other pin to determine the direction of rotation. This will change the DC and properly send it to the PWM signal. This also checks if the fan is on or off and won't do anything if it is off.	This is used to change the DC using the RPG any time it is turned
BTN_intr	This routine uses the pin change interrupt on the B register to get input from the button. When this interrupt is called it debounces the signal, checks if the interrupt was on a falling edge and then turns on or off the fan.	This is used for turning on and off the fan when the button is pressed
Fan	This routine will initialize Timer/Counter0 to be set in PWM mode with a frequency of 80kHz and initial duty cycle of 0. This will not turn the fan on but just begin the PWM signal generation so that when the fan gets turned on it is already initialized.	Will only be called once at the beginning of the code to initialize timer.

displayCString	This routine will display a string of characters to the LCD. It begins by setting register select high to let the LCD know it is sending characters. Each characters will be sent to the LCD in nibbles of data. The upper nibble will be sent first to port D and then strobe will be called to latch the nibble in place. After waiting 100us the lower nibble will be sent the same way to port D using strobe. The routine will continue to loop until all characters in the string have been sent by using a counter value.	Called every time a string of characters is sent to LCD. i.e. "DC ="
displayDC	This routine is used to break up the DC value into each of its individual decimal bits to be displayed on the screen. (i.e. 345 -> 3, 4, 5). It will load each individual number value into dtxt which is stored in RAM. To be able to separate the bits from each other, it uses a divide routine to divide by 10 which then leave the digit in the remainder value. After all the digits have been extracted from DC and placed into dtxt, dtxt will be loaded into the Z register to then be displayed.	Called every time the LCD is updated
displayDString	This routine is similar to displayCString in the fact that it also displays data to the LCD, but instead of displaying a string of characters it will display a string of data. This is useful since we store dtxt in memory. The same technique is used as displayCString by sending nibbles to the port, strobing, and delaying. The only difference is where the data is coming from	Called every time a DC value is being displayed
LCDInit	This routine initializes the LCD so that it is ready to display data. To do this it will first set the display on, clear the display, set the character entry mode, and set the cursor off. It will then wait 10 ms and then set the LCD to 8 bit, wait another 10 ms and set to 8 bit mode again, wait 200 us and set to 8 bit mode, wait another 200 us and then finally set to 4 bit mode. Then finally, it will wait 10 ms before setting the function as 2 line mode, turning the display off, clearing the display, waiting 10 ms again and then turning the LCD on. After all of this the LCD will finally be ready to use.	Only called once at the very beginning of execution
Command	This routine will let the LCD know that we want to send a command and then also send this command. To do this it first sets the register select as low, and then sends the nibbles of data to Port D.	This will be called anytime a command is being sent to LCD (i.e clear display)

Strobe	This routine strobes the enable pin on the LCD to be able to write data. It simply sets the enable as high, waits 1 us then sets it as low, then waits 100 us. This allows data to be sent to the LCD.	Called every time a command or data is being sent to LCD
--------	--	--

Timer Calculations:

To get a PWM frequency of 80 kHz:

$$80 \text{ kHz} = 16 \text{ kHz}/x$$

$$x = 200$$

Therefore, we need a prescaler of 1 and a TOP value of 199 to efficiently get a frequency of 80 kHz.

DC Calculations:

$$\text{DC \%} = (\text{DC} / 200) * 100$$

4. Conclusion

From this lab, we have gained a knowledge of LCD basics and how to send and display data on an LCD. We have also gained knowledge in PWM signal generation and how the duty cycle can be used to change this signal. Finally, we learned the importance of interrupts and how they can be used to handle certain interactions with the system.

5. Appendix A: Source Code

```

6.
7.
8.
9. .include "m328Pdef.inc"
10.
11. .dseg
12. dtxt: .BYTE 5
13.
14. .cseg
15. .def drem16uL = r10 ;remainder low
16. .def drem16uH = r11 ;remainder high
17. .def dd16uL = r12 ;dividend low
18. .def dd16uH = r13 ;dividend high
19. .def dres16uL = r12 ;result low
20. .def dres16uH = r13 ;result high
21. .def dv16uL = r14 ;divisor low
22. .def dv16uH = r15 ;divisor high
23. .def dcnt16u = r16 ;count for div
24.
25. .def timer = r17 ;used to count to 1 ms, set timer high to multiple of

```

```

26.
27. .def temp1 = r18      ;used to change dc
28. .def temp2 = r19
29.
30. .def fanReg = r20
31.
32. .def timer1 = r22
33. .def timer2 = r23
34.
35. .def dc = r24         ;duty cycle
36. .def discnt = r25
37.
38. .org 0x00
39.     rjmp start
40.
41. .org 0x06
42.     rjmp BTN_intr
43.
44. .org 0x08
45.     rjmp RPG_intr
46.
47. .org 0x1a
48.     ;constant strings to display
49.     msg1: .db "DC = ", 0x00
50.     msg2: .db "%                               Fan: ",0x00,0x00
51.     off:  .db "OFF"      ,0x00,0x00,0x00
52.     on:   .db "ON"       ,0x00,0x00,0x00,0x00
53.
54. start:
55.     ;Inputs and Ouputs
56.     cbi DDRB, 0 ;PB0: PushButton
57.
58.     cbi DDRC, 0 ;PB0: RPG Pin A
59.     cbi DDRC, 1 ;PB1: RPG Pin B
60.
61.     sbi DDRB, 4 ;PB4: LCD Enable
62.     sbi DDRB, 5 ;PB5: LCD Register Select
63.
64.     sbi DDRD, 0 ;PD0: D4
65.     sbi DDRD, 1 ;PD1: D5
66.     sbi DDRD, 2 ;PD2: D6
67.     sbi DDRD, 3 ;PD3: D7
68.
69.     sbi DDRD, 5 ;PD5: OC0B to send to fan
70.
71.     ldi timer, 10
72.     rcall delay_10ms
73.
74.     rcall LCDInit ;initialize LCD
75.
76.     ldi timer, 10
77.     rcall delay_10ms
78.
79.     ldi r30, low(2*msg1) ;load first message to display
80.     ldi r31, high(2*msg1)
81.     ldi discnt, 5        ;set counter to length of string
82.     rcall displayCString ;display "DC =" to the LCD
83.

```

```

84.    ldi temp1, low(000)    ;load the DC value
85.    mov dd16uL, temp1
86.    ldi temp1, high(000)  ;will change when know value
87.    mov dd16uH, temp1
88.    rcall displayDC       ;display the data to LCD
89.
90.    ldi r30, low(2*msg2)   ;load second message "% Fan"
91.    ldi r31, high(2*msg2)
92.    ldi discnt, 36         ;load the length of the message
93.    rcall displayCString   ;display to LCD
94.
95.    ldi r30, low(2*on)     ;set fan status to on
96.    ldi r31, high(2*on)
97.    ldi discnt, 2          ;set length of string to 2
98.    rcall displayCString   ;display fan status to LCD
99.
100.    lds temp1, PCICR
101.    ori temp1, 0x03
102.    sts PCICR, temp1
103.
104.    lds temp1, PCMSK0
105.    ori temp1, 0x01
106.    sts PCMSK0, temp1
107.
108.    lds temp1, PCMSK1
109.    ori temp1, 0x01
110.    sts PCMSK1, temp1
111.
112.    ldi dc, 0x20
113.
114.    rcall fan
115.
116.    sei
117.
118.    loop: ; waiting for interupts
119.        ldi timer, 30
120.        rcall delay_10ms
121.        rcall update_LCD_DC
122.        rjmp loop
123.
124.    update_LCD_DC:
125.        ldi temp1, 0x01
126.        rcall command
127.        sbi PORTB, 5
128.        ldi timer, 1
129.        rcall delay_10ms
130.        ldi r30, low(2*msg1) ;load first message to display
131.        ldi r31, high(2*msg1)
132.        ldi discnt, 5         ;set counter to length of string
133.        rcall displayCString ;display "DC =" to the LCD
134.
135.        ldi temp1, 100
136.        mul dc, temp1
137.
138.
139.        mov temp1, r0
140.        mov temp2, r1
141.

```



```

142.         ldi timer, 6
143.         div_64:
144.         clc
145.         ror temp2
146.         ror temp1
147.         dec timer
148.         brne div_64
149.
150.         ldi timer, 10
151.         mul temp1, timer
152.         push r0
153.         mov temp1, r1
154.         mul temp2, timer
155.         mov temp2, r0
156.         add temp2, temp1
157.         pop temp1
158.
159.         clc
160.         ror temp2
161.         ror temp1
162.         ror temp2
163.         ror temp1
164.
165.
166.         mov dd16uL, temp1      ;load the DC value
167.         mov dd16uH, temp2
168.         rcall displayDC        ;display the data to LCD
169.
170.         ldi r30, low(2*msg2)   ;load second message "% Fan"
171.         ldi r31, high(2*msg2)
172.         ldi discnt, 36         ;load the length of the message
173.         rcall displayCString   ;display to LCD
174.
175.         cpi fanReg, 0x00
176.         breq display_off
177.
178.         ldi r30, low(2*on)      ;set fan status to off
179.         ldi r31, high(2*on)
180.         ldi discnt, 2          ;set length of string to 2
181.         rcall displayCString    ;display fan status to LCD
182.         ret
183.         display_off:
184.         ldi r30, low(2*off)     ;set fan status to off
185.         ldi r31, high(2*off)
186.         ldi discnt, 3          ;set length of string to 2
187.         rcall displayCString    ;display fan status to LCD
188.         ret                    ;go back to main loop and
                                ;wait for button press or RPG
189.
190.
191.         RPG_intr:
192.         push temp1
193.         in temp1, sreg
194.         push temp1
195.         push timer1
196.         push timer2
197.         push timer
198.

```

```

199.         ldi timer, 20
200.         RPG_delay:
201.         rcall delay100us
202.         dec timer
203.         brne RPG_delay
204.
205.         in temp1, PCIFR
206.         ori temp1, 0x02
207.         out PCIFR, temp1
208.
209.         cpi fanReg, 0x00
210.         breq RPG_off
211.
212.         sbic PINC, 0
213.         rjmp RPG_end
214.
215.         sbis PINC, 1
216.         rjmp decrement
217.
218.         increment:
219.         ldi temp1, 2
220.         add dc, temp1
221.         brcc RPG_end
222.         ldi dc, 0xFF
223.         rjmp RPG_end
224.
225.         decrement:
226.         ldi temp1, 2
227.         sub dc, temp1
228.         brcc RPG_end
229.         ldi dc, 0x00
230.         rjmp RPG_end
231.
232.         RPG_end:
233.         out OCR0B, dc
234.         RPG_off:
235.
236.         pop timer
237.         pop timer2
238.         pop timer1
239.         pop temp1
240.         out sreg, temp1
241.         pop temp1
242.         reti
243.
244.         BTN_intr:
245.         push temp1
246.         in temp1, sreg
247.         push temp1
248.         push temp2
249.         push timer1
250.         push timer2
251.         push timer
252.
253.         ldi timer, 1
254.         rcall delay_10ms      ; wait for noise
255.
256.         in temp1, PCIFR      ; clear interrupt flag

```

```

257.         ori temp1, 0x01
258.         out PCIFR, temp1
259.
260.         sbic PINB, 0           ; falling edge
261.         rjmp BTN_skip
262.
263.         ldi temp1, 0x01
264.         eor fanReg, temp1
265.         breq BTN_off
266.
267.         out OCR0B, dc
268.         rjmp BTN_skip
269.     BTN_off:
270.         ldi temp1, 0x00
271.         out OCR0B, temp1
272.         rjmp BTN_skip
273.
274.     BTN_skip:
275.         pop timer
276.         pop timer2
277.         pop timer1
278.         pop temp2
279.         pop temp1
280.         out sreg, temp1
281.         pop temp1
282.         reti
283.
284.     fan:
285.         ldi temp1, (1<<WGM01)|(1<<WGM00)    ; Fast PWM
286.         ldi temp2, (1<<COM0B1)              ; Clear OC0B on match
287.         out TCCR0A, temp1
288.         or temp1, temp2
289.         out TCCR0A, temp1
290.
291.         ldi fanReg, 0x01
292.
293.         ldi temp1, (1<<CS00)                  ; No prescaler (F_CPU/1)
294.         out TCCR0B, temp1
295.
296.         ldi temp1, 0
297.         out OCR0B, temp1
298.         ret
299.
300.     displayCString: ;displays a string of characters
301.         L20: ;loops through every individual character
302.             sbi PORTB, 5 ;set register select to read from data register
303.             lpm          ;load byte pointed to by Z register
304.             tst r0       ;if no value to display go to done_str
305.             breq done_str
306.             swap r0      ;upper nibble in place
307.             out PORTD, r0 ;send to LCD
308.             rcall strobe ;latch nibble
309.             rcall delay100us ;wait 100us
310.             swap r0      ;lower nibble in place
311.             out PORTD, r0 ;send to LCD
312.             rcall strobe ;latch nibble
313.             rcall delay100us ;wait 100 us
314.             adiw Z, 1 ;increment Z pointer to next byte

```

```

315.             dec discnt      ;decrement string length counter
316.             brne displayCString ;continue to loop until end of string
317. done_str:
318.             ret
319.
320. displayDC: ;displays a DC percent value
321.             ldi temp1, 0x00   ;terminating null character
322.             sts dtxt+4, temp1 ;store in RAM
323.
324.             rcall divide      ;get LSB from duty cycle
325.             ldi temp1, 0x30
326.             add drem16uL, temp1 ;offset to ASCII character
327.             sts dtxt+3,drem16uL ;store LSB in RAM
328.
329.             ldi temp1, 0x2e    ;load in decimal point
330.             sts dtxt+2,temp1   ;store in RAM
331.
332.             mov dd16uL, dres16uL ;set dividend to last result
333.             mov dd16uH, dres16uH
334.             rcall divide      ;get the next value
335.             ldi temp1, 0x30
336.             add drem16uL, temp1 ;offset to ASCII
337.             sts dtxt+1, drem16uL ;store in RAM
338.
339.             ldi temp1, 0x30
340.             add dres16uL, temp1 ;dres holds final value
341.             sts dtxt+0, dres16uL ;store in RAM
342.
343.             ldi r30, low(dtxt) ;load in the created string
344.             ldi r31, high(dtxt)
345.             rcall displayDString ;display data string
346.             ret
347.
348. displayDString: ;displays a data string, containing DC value
349.             ld r0, Z+
350.             tst r0            ;reached end of message?
351.             breq done_DString ;if yes -> quit
352.             swap r0           ;upper nibble in place
353.             out PORTD, r0     ;send upper nibble out
354.             rcall strobe      ;latch nibble
355.             rcall delay100us  ;delay 100 us
356.             swap r0           ;lower nibble in place
357.             out PORTD, r0     ;send lower nibble
358.             rcall strobe      ;latch nibble
359.             rcall delay100us  ;delay 100 us
360.             rjmp displayDString
361. done_DString:
362.             ret
363.
364. LCDInit: ;Initialize LCD
365.             ldi temp1, 0x08    ;sets display on
366.             rcall command      ;send command to LCD
367.
368.             ldi temp1, 0x01    ;clear display
369.             rcall command      ;send command to LCD
370.
371.             ldi temp1, 0x06    ;character entry mode set
372.             rcall command      ;send command to LCD

```

```

373.
374.     ldi temp1, 0x0C      ;sets display on, and cursor off
375.     rcall command       ;send command to LCD
376.
377.     ldi timer, 10
378.     rcall delay_10ms    // delay 100 ms
379.
380.     cbi PORTB, 5        ;set enable low
381.     ldi temp1, 0x03     ;set to 8 bit mode
382.     out PORTD, temp1    ;send to LCD
383.     rcall strobe
384.
385.     ldi timer, 1
386.     rcall delay_10ms
387.
388.     ldi temp1, 0x03
389.     out PORTD, temp1    ;set to 8 bit mode
390.     rcall strobe       ;send to LCD
391.
392.     rcall delay100us
393.     rcall delay100us    ;delay 200 us (delay2 * 2 = 200 us)
394.
395.     ldi temp1, 0x03
396.     out PORTD, temp1    ;set to 8 bit mode for final time
397.     rcall strobe       ;send to LCD
398.
399.     rcall delay100us    ;delay 200 us
400.     rcall delay100us
401.
402.     ldi temp1, 0x02
403.     out PORTD, temp1    ;set to 4 bit mode
404.     rcall strobe       ;send to LCD
405.
406.     ldi timer, 1
407.     rcall delay_10ms
408.
409.     ldi temp1, 0x28     ;function set: 2 line mode
410.     rcall command       ;send command to LCD
411.
412.     ldi temp1, 0x08     ;set display off
413.     rcall command       ;send command to LCD
414.
415.     ldi temp1, 0x01     ;clear display
416.     rcall command       ;send command to LCD
417.
418.     ldi timer, 1
419.     rcall delay_10ms
420.
421.     ldi temp1, 0x06     ;set character entry mode
422.     rcall command       ;send command to LCD
423.
424.     ldi temp1, 0x0C     ;set display on and cursor off
425.     rcall command       ;send command to LCD
426.     ret
427.
428. command: ;sends command to LCD
429.     cbi PORTB, 5        ;set register select to read from inst. register
430.     swap temp1          ;upper nibble in place

```

```

431.         out PORTD, temp1 ;send to LCD
432.         rcall strobe    ;latch nibble
433.         swap temp1      ;lower nibble in place
434.         out PORTD, temp1 ;send to LCD
435.         rcall strobe    ;latch nibble
436.         ret
437.
438. strobe:   ;strobes enable on LCD
439.         sbi PORTB, 4    ;sets enable high
440.         nop
441.         nop
442.         nop             ;1 us delay
443.         nop
444.         nop
445.         cbi PORTB, 4    ;sets enable low
446.         rcall delay100us
447.         ret
448.
449.
450. delay_10ms:
451.         rcall delay_long
452.         dec timer
453.         breq delay_10ms_end
454.         rjmp delay_10ms
455. delay_10ms_end:
456.         ret
457.
458. .equ count = 0x9A ; 49 ms @ 16 MHz --> this can likely be shorter and
459. ; needs to be modified as needed
460. delay_long:
461.         ldi timer2, count
462.         d1:
463.         ldi timer1, 0xff ; r29 <-- load a 8-bit value into counter register
for inner loop
464.         d2:
465.         nop
466.         dec timer1
467.         brne d2
468.         subi timer2, 1
469.         brne d1
470.         ret
471.
472.
473. delay100us: ;delay function from Lab1
474.         ldi timer1, 0xF ;changed registers and some formatting
475.         d3:
476.         ldi timer2, 0x1B ;100 u s
477.         d4:
478.         nop
479.         dec timer2
480.         brne d4
481.         dec timer1
482.         brne d3
483.         ret
484.
485. divide: ;divides 16 bit number by 10, to allow us to seperate DC values
486.         clc ;clear all registers that will be used
487.         clr drem16uL

```

```

488.      clr drem16uH
489.      ldi temp1, low(10)           ;will always divide by 10
490.      mov dv16uL, temp1
491.      ldi temp1, high(10)          ;so set dividend low/high to 10
492.      mov dv16uH, temp1
493.      div16u:
494.          clr drem16uL              ;clear remainder Low byte
495.          sub drem16uH,drem16uH     ;clear remainder High byte and
carry
496.          ldi dcnt16u,17            ;init loop counter
497.      d16u_1:
498.          rol dd16uL                ;shift left dividend
499.          rol dd16uH
500.          dec dcnt16u               ;decrement counter
501.          brne d16u_2              ;if done
502.          ret                       ;return
503.      d16u_2:
504.          rol drem16uL              ;shift dividend into remainder
505.          rol drem16uH
506.          sub drem16uL,dv16uL       ;remainder = remainder -
divisor
507.          sbc drem16uH,dv16uH
508.          brcc d16u_3              ;if result negative
509.          add drem16uL,dv16uL       ;restore remainder
510.          adc drem16uH,dv16uH
511.          clc                       ;clear carry to be shifted
into result
512.          rjmp d16u_1              ;else
513.      d16u_3:
514.          sec                       ;set carry to be shifted
into result
515.          rjmp d16u_1

```

516. Appendix B: References

- ❖ AVR200.zip divide routines
- ❖ Code snippets from lecture slides